

Pure Borrow

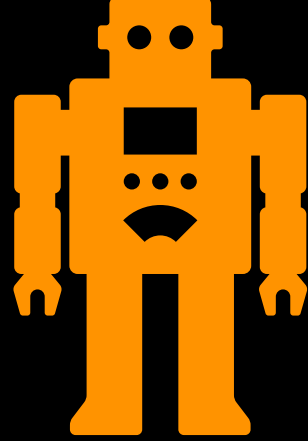
Linear Haskell Meets Rust-Style Borrowing

Yusuke Matsushita Kyoto University

Hiromi Ishii JIJ Inc.

June 19, 2026 — PLDI '26 @ Boulder, USA

Big picture

Computers 

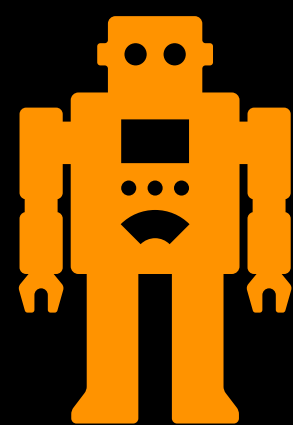
Imperative

Software



Functional

Humans 



C / C++

➤ Ownership types



Safe & Robust

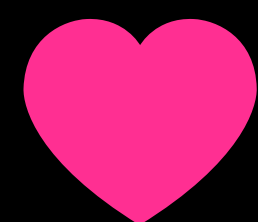
Rust

Borrows



Mutation w/o
ownership passing

Our work



Pure Borrow

➤ Borrows !

- Pure & Mutate &
Parallelize *Parallel quicksort*
- Just APIs
- Metatheory *WIP* *Purity by history*

Linear



Linear types



Mutate



Haskell

Purity!



Same result
regardless of timing

Parallelism
Lazy evaluation

High-level comparison

	Pure 	Mutate	Borrows 	Parallel	Leak-free
Pure only	●	○		●	
ST monad	●	●	○	○	○
Rust	○	●	●	●	○
Basic Linear Haskell	●	●	○	●	●
Our work Pure Borrow	●	●	●	●	●

More technically

Background **Linear Haskell** [Bernardy+ '18]

Original **Haskell** Persistent data $\rightarrow$ **Copy**

$\text{Vec } a \rightarrow \text{Int} \rightarrow (a \rightarrow a) \rightarrow \text{Vec } a$

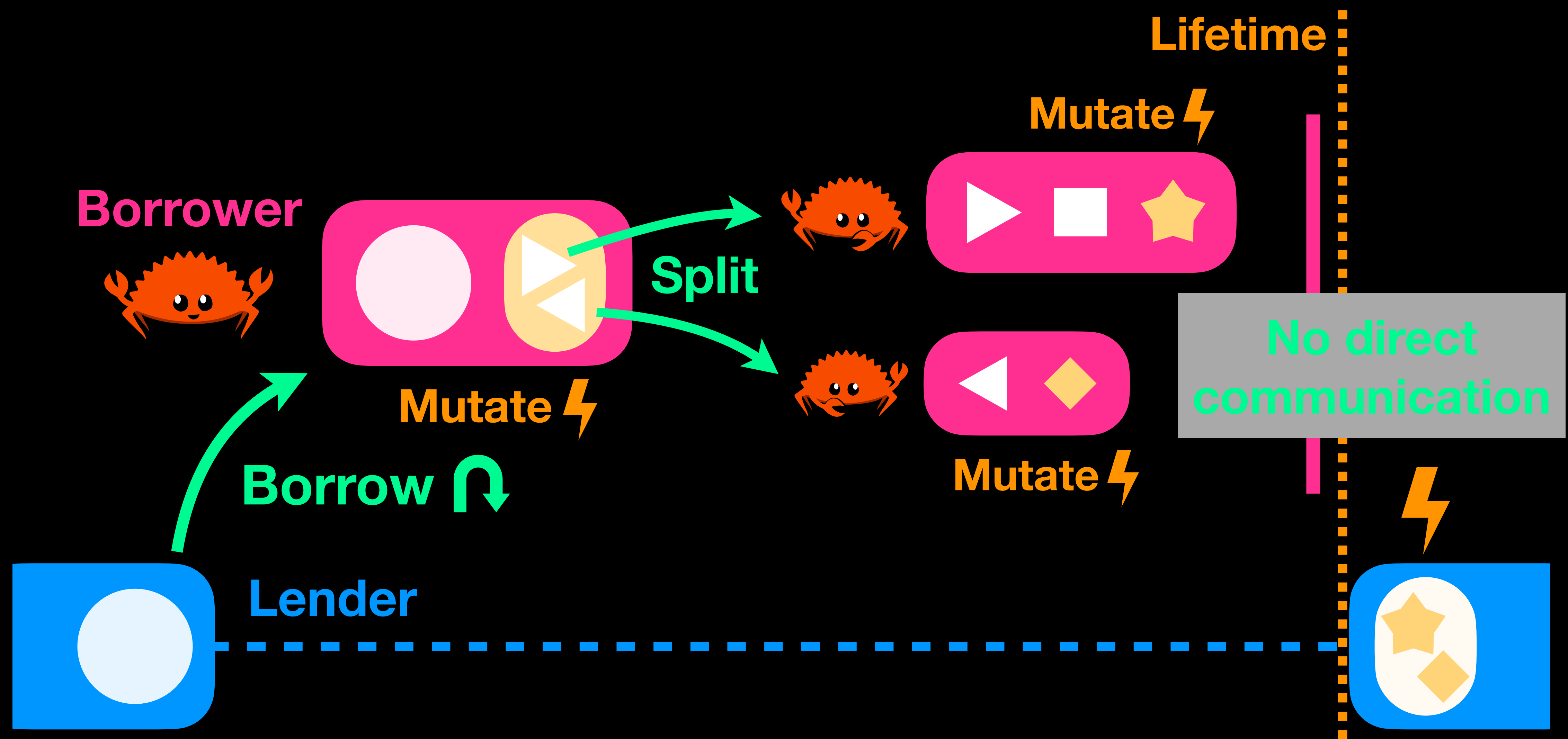

[0, 1, 2, ..., 999] 0 (+ 7) [7, 1, 2, ..., 999]

Linear arrow Argument used exactly once

Vec a $\multimap$ Int $\rightarrow$ (a $\multimap$ a) $\multimap$ Vec a **Ownership passing**

Linear Haskell Linear $\rightarrow$ **Mutate in place!**

Background Rust-style borrowing [Matsakis+ '14]



Our work **Pure Borrow** **Just APIs!**

Borrower Lender

Borrow $\text{borrow} :: \text{Lin} \multimap a \multimap (\text{Mut}^\alpha a, \text{Lend}^\alpha a)$

Linearity witness [Spiwack+ '22]

BO Monad

Mutate ⚡ $\text{modifyAt} :: \text{Mut}^\alpha (\text{Vec } a) \multimap \text{Int} \rightarrow (a \multimap a) \multimap \text{BO}^\alpha (\text{Mut}^\alpha (\text{Vec } a))$

Split $\text{splitAt} :: \text{Mut}^\alpha (\text{Vec } a) \multimap \text{Int} \rightarrow (\text{Mut}^\alpha (\text{Vec } a), \text{Mut}^\alpha (\text{Vec } a))$

Drop $\text{drop} :: \text{Mut}^\alpha a \multimap ()$

α has ended

Reclaim $\text{reclaim} :: \text{End}^\alpha \Rightarrow \text{Lend}^\alpha a \multimap a$ ⚡

**No direct communication
from borrowers to the lender**

cf. **RustBelt** [Jung+ '18]

Key idea BO Monad

$\text{BO}^\alpha a$ = **Computation** of a allowed during the **lifetime** α

Timing constraint $t(\text{Borrower access}) < t(\text{Lender reclaim})$

Pure  $\text{runBO} :: \text{Lin} \multimap (\forall \alpha. \text{BO}^\alpha (\text{End}^\alpha \Rightarrow a)) \multimap a$
Rank-2 poly

cf. **ST Monad** [Launchbury+ '94] $\text{runST} :: (\forall s. \text{ST}^s a) \rightarrow a$

Parallel $\text{parBO} :: \text{BO}^\alpha a \multimap \text{BO}^\alpha b \multimap \text{BO}^\alpha (a, b)$

Pure $\leftarrow$ **Linearity** guarantees **disjointness**

Share & Reborrow

Temporarily share

Shared borrower

$\text{sharing} :: \text{Mut}^\alpha a \multimap (\forall \beta. \text{Share}^{\beta \wedge \alpha} a \xrightarrow{\text{Non-linear}} \text{BO}^{\beta \wedge \alpha'} b) \multimap \text{BO}^{\alpha'} (b, \text{Mut}^\alpha a)$

Lifetime intersection

$\uparrow \text{share} :: \text{Mut}^\alpha a \multimap \text{Ur} (\text{Share}^\alpha a) + \text{reborrowing}$

Unrestricted = Non-linear

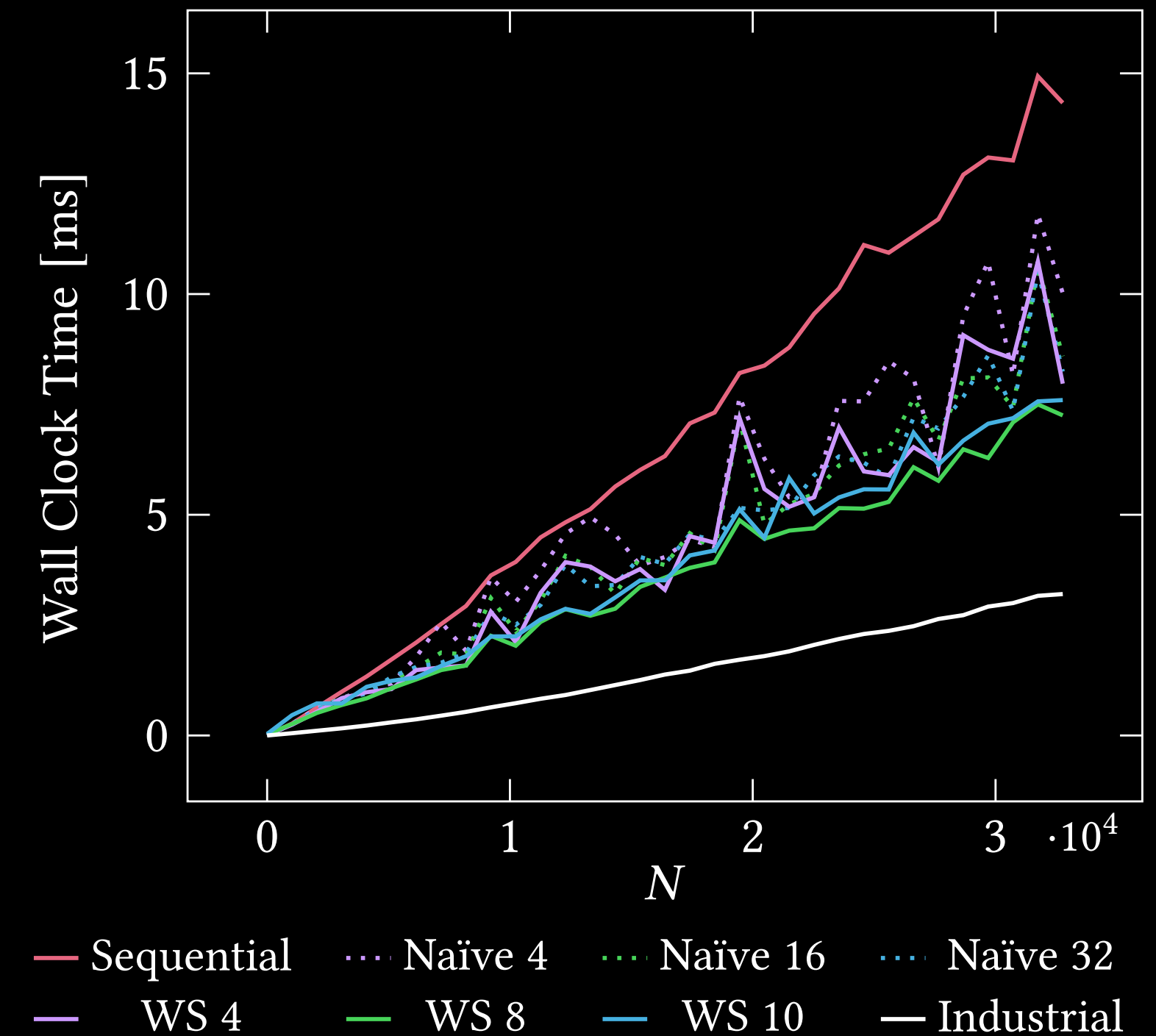
Reborrow

$\text{reborrowing} :: \text{Mut}^\alpha a \multimap (\forall \beta. \text{Mut}^{\beta \wedge \alpha} a \multimap \text{BO}^{\beta \wedge \alpha'} b) \multimap \text{BO}^{\alpha'} (b, \text{Mut}^\alpha a)$

Case study Parallel quicksort

```
qsort :: Mutα (Vec Int) → BOα ()
qsort v = let (Ur n, v) = size v in
  if n ≤ 1 then pure (drop v) else do
    (Ur a, v) ← readAt (n // 2) v
    (i, v) ← pivotSwap a v Mutate! ⚡
    let (lo, hi) = splitAt v i Split!
    drop <$> parBO (qsort lo) (qsort hi)
    Parallelize!
```

Also: **Work-stealing scheduler**



Safe parallelized mutation
by Pure Borrow!

Metatheory Purity of borrowing

- ▶ Challenge: Justify the lender reclaim?
- ▶ Key idea: History-based operational semantics
 - BO monad can record the past mutation histories! → $\text{End}^\alpha a$
- ▶ Theorem: History-based op. sem. is strongly confluent
 - Referential transparency: Future work, due to borrow id generation
- ▶ WIP: Correspondence with standard op. sem.
 - Roughly sketched progress & bisimulation for a dynamic, relational linear type system

Related work **OxCaml** [Lorenzen+ '24, Georges+ '25]

- ▶ **OCaml** + **Ownership** etc. → **Safe low-level** controls
- ▶ **Lightweight mode** annotation
 - Coarse **lifetime modes**: **local** vs **global**
- ▶ **No purity**
- ▶ **Linear** vs **Unique** [Marshall+ '22]

comonad
many $\leq$ **once**

$\text{Ur } a \multimap a$

$\text{Ur } a \multimap \text{Ur } (\text{Ur } a)$

unique $\leq$ **aliased monad**

$a \multimap (\text{Mut}^\alpha a, \text{Lend}^\alpha a)$

$\text{Mut}^\alpha (\text{Mut}^\beta a) \multimap \text{Mut}^{\alpha \wedge \beta} a$

Pure Borrow is already there!

 Hackage :: [Package]

Search

· Browse · What's new · Upload · User

pure-borrow: Rust-style borrowing in Linear Haskell with purity

[[bsd3](#), [library](#), [linear-haskell](#), [program](#)] [[Propose Tags](#)] [[Report a vulnerability](#)]

This package realizes rust-style borrowing in Linear Haskell with purity and concurrency support. See [Control.Monad.Borrow.Pure](#) for the main API documentation, and see our paper *Pure Borrowing: Linear Haskell Meets Rust-Style Borrowing* by Y. Matsushita and H. Ishii for the details.

[\[Skip to README\]](#)

Build BuildFailed Documentation Available

Modules

[[Index](#)] [[Quick Jump](#)]

Control

- Concurrent*
 - DivideConquer*
 - [Control.Concurrent.DivideConquer.Linear](#)
- STM*
 - [Control.Concurrent.STM.TMDeque](#)
 - [Control.Concurrent.STM.TMDequeRingBuffer](#)
- Monad*
 - Borrow*

Versions

[\[RSS\]](#)
0.0.0.0

Change log

[CHANGELOG.md](#)

Dependencies

[array](#), [base](#) ([>=4.17](#) & [<5](#)), [bytestring](#), [cassava](#), [containers](#), [deepseq](#), [directory](#), [file-embed](#), [hybrid-vectors](#), [linear-base](#) ([>=0.7](#)), [linear-generics](#), [monoidal-containers](#), [optparse-applicative](#), [process](#), [pure-borrow](#), [random](#), [stm](#), [tasty](#), [tasty-bench](#), [template-haskell](#), [temporary](#), [text](#), [transformers](#), [unordered-containers](#), [vector](#), [vector-algorithms](#) [[details](#)]

Tested with

ghc ==9.10.2 || ==9.12.4 || ==9.14.1

License

[BSD-3-Clause](#)

Copyright

Copyright (c) 2025-present, Yusuke Matsushita and Hiromi Ishii

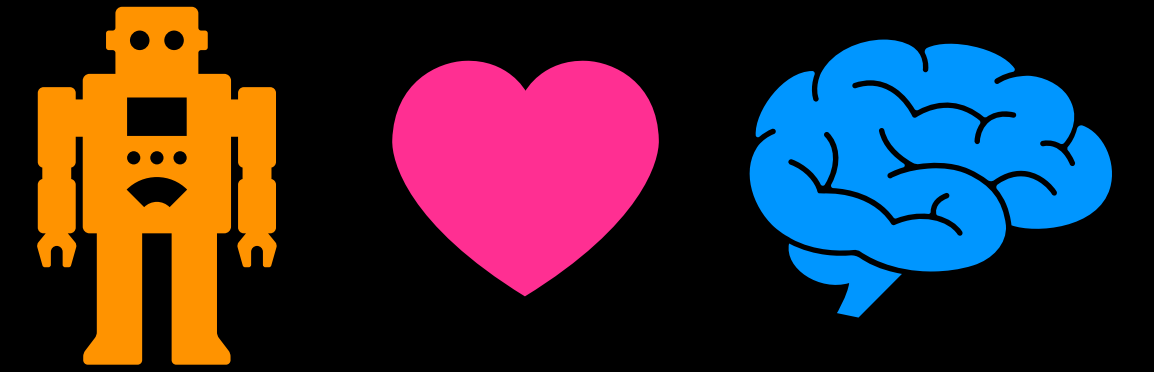
Author

Yusuke Matsushita and Hiromi Ishii

Pure Borrow works on
GHC w/o tweaks

Give it a try!

Pure Borrow — Summary



- ▶ **Linear Haskell** + **Rust**-style **borrowing**
 - **Pure** & **Mutate** w/o **ownership passing** & **Parallelize**
- ▶ **BO monad** — **Lifetime** constraint for **borrows**
 - Like **ST monad**, but with **linearity** & **parallelization**
- ▶ **Case study** — **Parallel quicksort**
- ▶ **Metatheory** — **Purity** by **history**-based **borrow** model

Thank you!